WEB TECHNOLOGY (EXPERIMENT 2 TO 12) CODES

Experiments 2–6: HTML and CSS

Experiment 2 — Simple Blog Webpage

Concept: Headings, paragraphs, lists, tables, hyperlinks and external CSS.

index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>My Personal Blog</title>
    <link rel="stylesheet" href="styles.css">
</head>
<body>
    <h1>My Personal Blog</h1>
    <p class="center">Welcome to my blog!</p>
    <hr>

    <h2>About Me</h2>
    <p>I am a Computer Science student interested in programming.
</p>

    <h2>My Skills</h2>
    <ul>
        <li>HTML</li>
        <li>CSS</li>
        <li>JavaScript</li>
    </ul>

    <h2>Education</h2>
    <table>
        <tr><th>Course</th><th>Status</th></tr>
        <tr><td>B.E. CSE</td><td>Studying</td></tr>
    </table>

    <h2>Daily Routine</h2>
    <ol>
        <li>Attend college</li>
        <li>Practise coding</li>
        <li>Study</li>
    </ol>

    <h2>Favourite Website</h2>
    <a href="https://www.wikipedia.org">Wikipedia</a>

    <h2>Favourite Quote</h2>
    <p><i>Learning never stops.</i></p>

    <h2>Contact</h2>
    <p>Email: student@example.com</p>
</body>
</html>
```

styles.css

```
body {
    font-family: Arial;
    margin: 25px;
    color: black;
    background: white;
}

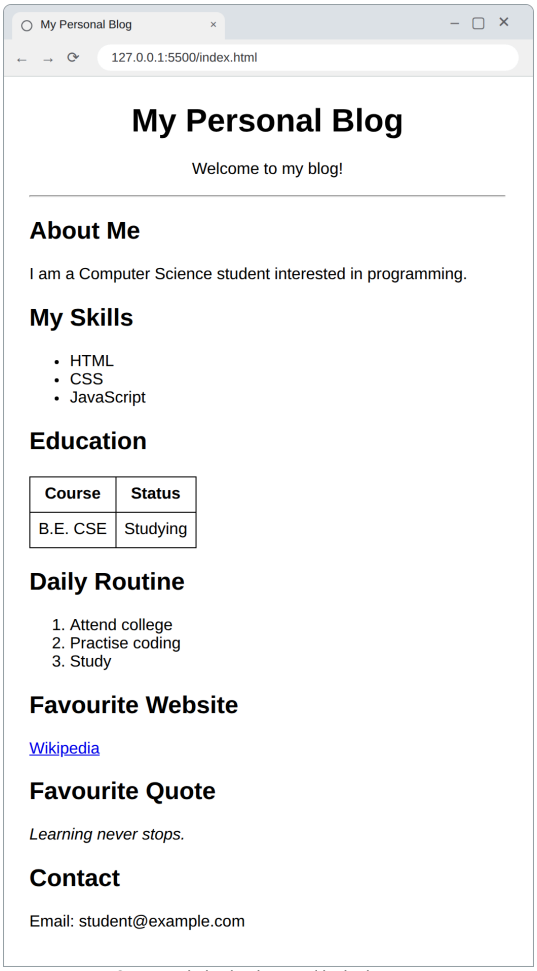
h1, .center {
    text-align: center;
}

table {
    border-collapse: collapse;
}

th, td {
    border: 1px solid black;
    padding: 8px;
}
```

Understand: The `<link>` tag connects the HTML file to the external CSS file. The class `center` lets CSS align a particular paragraph without affecting every paragraph.

OUTPUT



Output — index.html opened in the browser

Experiment 3 — Hotspot Creation

Concept: An image or diagram containing clickable regions that open different webpages.

To keep the program simple and self-contained, this version uses an SVG illustration with clickable regions. In an exam question specifically requiring HTML image maps, use `<map>` and `<area>` instead.

index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Clickable Hotspots</title>
    <link rel="stylesheet" href="styles.css">
</head>
<body>
    <h1>Clickable Hotspots</h1>
    <p>Click a region to open its information page.</p>

    <svg width="360" height="300" viewBox="0 0 360 300">
        <rect x="1" y="1" width="358" height="298"
              fill="white" stroke="black"/>

        <path d="M150 25 L190 45 L210 80 L195 110
                 L220 140 L205 180 L180 215 L155 265
                 L130 230 L115 190 L95 155 L120 120
                 L105 85 L130 55 Z"
              fill="white" stroke="black" stroke-width="2"/>

        <a href="https://en.wikipedia.org/wiki/Delhi"
           target="_blank">
            <circle cx="155" cy="105" r="9"
                    fill="white" stroke="black"/>
            <text x="170" y="109" font-size="12">Delhi</text>
        </a>

        <a href="https://en.wikipedia.org/wiki/Tamil_Nadu"
           target="_blank">
            <circle cx="165" cy="225" r="9"
                    fill="white" stroke="black"/>
```

```
            <text x="180" y="229" font-size="12">Tamil Nadu</text>
        </a>
    </svg>
</body>
</html>
```

styles.css

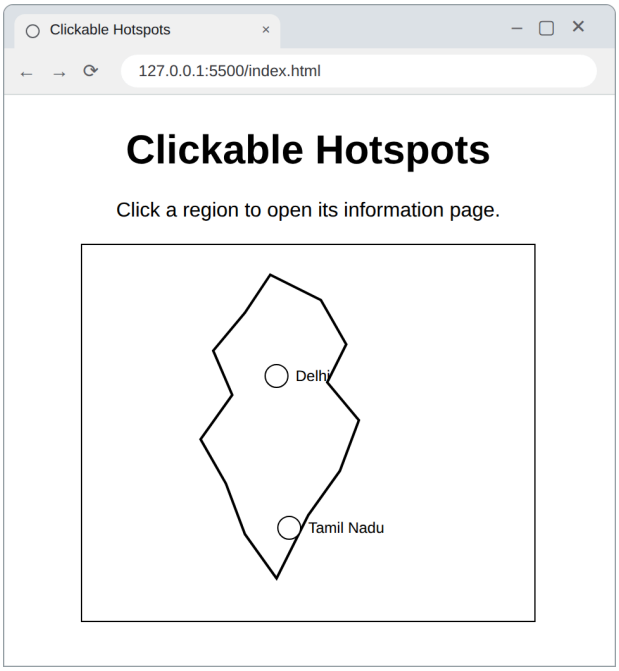
```
body {
    font-family: Arial;
    color: black;
    background: white;
    margin: 25px;
}

h1, p {
    text-align: center;
}

svg {
    display: block;
    margin: auto;
    max-width: 100%;
}
```

Understand: The `<a>` element makes the SVG region clickable. `href` specifies the destination, while `target="_blank"` opens it in a new tab. This is an illustrative diagram, not a geographically accurate India map.

OUTPUT



Output — clickable SVG hotspots (Delhi and Tamil Nadu open Wikipedia pages)

Experiment 4 — Registration Form

Concept: Forms, input types, labels, dropdowns, radio/checkbox controls and basic validation.

index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Registration Form</title>
    <link rel="stylesheet" href="styles.css">
</head>
<body>
    <h1>Registration Form</h1>

    <form id="form">
        <label>Full Name</label>
        <input type="text" required>

        <label>Date of Birth</label>
        <input type="date" required>

        <label>Email</label>
        <input type="email" required>

        <label>Mobile Number</label>
        <input type="tel" pattern="[0-9]{10}" required>

        <label>Gender</label>
        <select required>
            <option value="">Select Gender</option>
            <option>Male</option>
            <option>Female</option>
            <option>Other</option>
        </select>

        <label>Address</label>
        <textarea required></textarea>

        <label>State</label>
        <select>
            <option>Tamil Nadu</option>
            <option>Kerala</option>
            <option>Karnataka</option>
        </select>

        <p>
            <input type="checkbox" required>
            I confirm the details are correct.
        </p>

        <button type="submit">Register</button>
        <button type="reset">Reset</button>
    </form>

    <p id="message"></p>

    <script>
        document.getElementById("form").onsubmit = function(event)
{
            event.preventDefault();
            document.getElementById("message").textContent =
                "Registration successful!";
        };
    </script>
</body>
</html>
```

styles.css

```
body {
    font-family: Arial;
    margin: 25px;
    color: black;
    background: white;
}

h1 {
    text-align: center;
}

form {
    max-width: 450px;
    margin: auto;
}

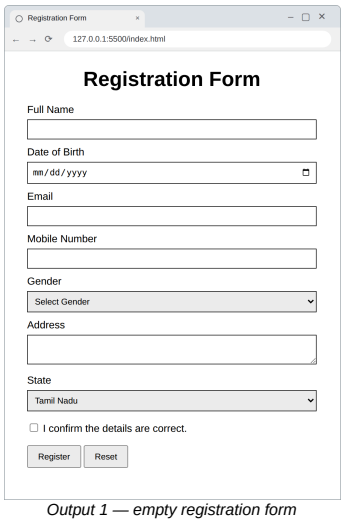
label {
    display: block;
    margin-top: 12px;
}

input:not([type="checkbox"]),
select,
textarea {
    box-sizing: border-box;
    width: 100%;
    padding: 7px;
    border: 1px solid black;
    margin-top: 5px;
}

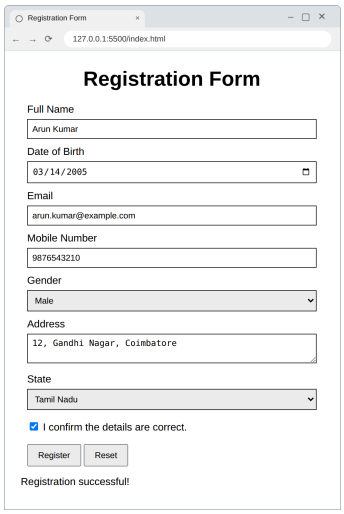
button {
    padding: 8px 15px;
}
```

Understand: `required` prevents empty fields from being submitted. `type="email"` checks the email format, and `pattern="[0-9]{10}"` requires exactly ten digits. JavaScript's `preventDefault()` stops the browser from reloading the page after submission.

OUTPUT



Output 1 — empty registration form



Output 2 — filled form after clicking Register

Experiment 5 — College Website Using HTML and CSS

Concept: Multiple webpages connected by a common navigation bar.

Create this structure:

```
Ex05/
    index.html
    events.html
    admission.html
    fees.html
    results.html
    contact.html
    styles.css
```

index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>ABC College - Home</title>
    <link rel="stylesheet" href="styles.css">
</head>
<body>
    <header>
        <h1>ABC College</h1>
        <p>Education and Excellence</p>
    </header>

    <nav>
        <a href="index.html">Home</a>
        <a href="events.html">Events</a>
        <a href="admission.html">Admission</a>
        <a href="fees.html">Fees</a>
        <a href="results.html">Results</a>
        <a href="contact.html">Contact</a>
    </nav>

    <main>
        <h2>Welcome to ABC College</h2>
        <p>Providing education in engineering and technology.</p>
        <a href="admission.html">Apply Now</a>
    </main>

    <footer>ABC College</footer>
</body>
</html>
```

events.html

```
<!DOCTYPE html>
<html>
<head>
    <title>College Events</title>
    <link rel="stylesheet" href="styles.css">
</head>
<body>
    <h1>ABC College</h1>
    <nav>
        <a href="index.html">Home</a>
        <a href="events.html">Events</a>
        <a href="admission.html">Admission</a>
        <a href="fees.html">Fees</a>
        <a href="results.html">Results</a>
        <a href="contact.html">Contact</a>
    </nav>

    <h2>Upcoming Events</h2>
    <ul>
        <li>Technical Symposium</li>
        <li>Workshop</li>
        <li>Sports Day</li>
    </ul>
</body>
</html>
```

admission.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Admissions</title>
    <link rel="stylesheet" href="styles.css">
</head>
<body>
    <h1>ABC College</h1>
    <nav>
        <a href="index.html">Home</a>
        <a href="events.html">Events</a>
        <a href="admission.html">Admission</a>
        <a href="fees.html">Fees</a>
        <a href="results.html">Results</a>
        <a href="contact.html">Contact</a>
    </nav>

    <h2>Admission Information</h2>
    <p>Applicants must satisfy the required eligibility criteria.
</p>
    <p>Contact the college office to apply.</p>
</body>
</html>
```

fees.html

```
<!DOCTYPE html>
<html>
<head>
    <title>College Fees</title>
    <link rel="stylesheet" href="styles.css">
</head>
<body>
    <h1>ABC College</h1>
    <nav>
        <a href="index.html">Home</a>
        <a href="events.html">Events</a>
        <a href="admission.html">Admission</a>
        <a href="fees.html">Fees</a>
        <a href="results.html">Results</a>
        <a href="contact.html">Contact</a>
    </nav>

    <h2>Fee Structure</h2>
    <table>
        <tr><th>Course</th><th>Annual Fee</th></tr>
        <tr><td>CSE</td><td>80000</td></tr>
        <tr><td>ECE</td><td>75000</td></tr>
    </table>
</body>
</html>
```

results.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Results</title>
    <link rel="stylesheet" href="styles.css">
</head>
<body>
    <h1>ABC College</h1>
    <nav>
        <a href="index.html">Home</a>
        <a href="events.html">Events</a>
        <a href="admission.html">Admission</a>
        <a href="fees.html">Fees</a>
        <a href="results.html">Results</a>
        <a href="contact.html">Contact</a>
    </nav>

    <h2>Student Results</h2>
    <table>
        <tr><th>Name</th><th>Mark</th></tr>
        <tr><td>Arun</td><td>85</td></tr>
        <tr><td>Kavi</td><td>90</td></tr>
    </table>
</body>
</html>
```

contact.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Contact Us</title>
    <link rel="stylesheet" href="styles.css">
</head>
<body>
    <h1>ABC College</h1>
    <nav>
        <a href="index.html">Home</a>
        <a href="events.html">Events</a>
```

```
        <a href="admission.html">Admission</a>
        <a href="fees.html">Fees</a>
        <a href="results.html">Results</a>
        <a href="contact.html">Contact</a>
    </nav>

    <h2>Contact Us</h2>
    <p>Coimbatore, Tamil Nadu</p>
    <p>Email: college@example.com</p>

    <form onsubmit="event.preventDefault(); alert('Feedback
submitted');">
        <label>Name</label>
        <input type="text" required>

        <label>Message</label>
        <textarea required></textarea>

        <button type="submit">Send</button>
    </form>
</body>
</html>
```

styles.css — Shared by all six pages

```
body {
    font-family: Arial;
    margin: 0;
    color: black;
    background: white;
}

header, footer {
    text-align: center;
    padding: 15px;
    border: 1px solid black;
}

nav {
    text-align: center;
    padding: 12px;
    border-bottom: 1px solid black;
}

nav a {
    color: black;
    margin: 5px;
}

main {
    margin: 25px;
}

table {
    border-collapse: collapse;
}

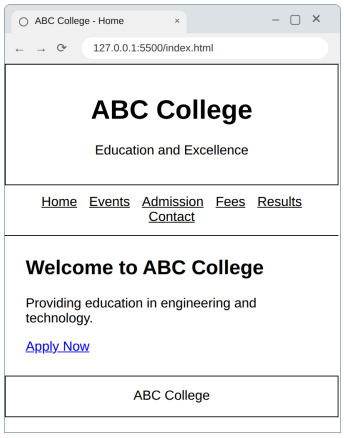
th, td {
    border: 1px solid black;
    padding: 8px;
}

label {
    display: block;
    margin: 10px 0;
}

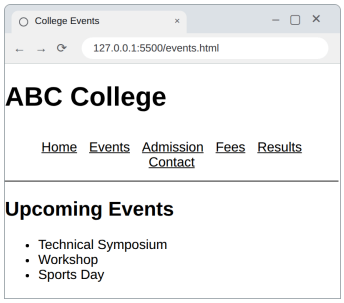
input, textarea {
    display: block;
    margin-top: 5px;
    padding: 7px;
}
```

Understand: The `href` attribute connects pages. The same CSS file gives every page a consistent appearance. You can add more pages by copying the navigation bar and changing the page content.

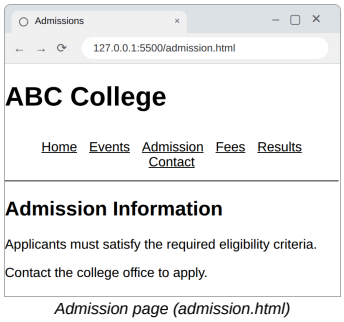
OUTPUT



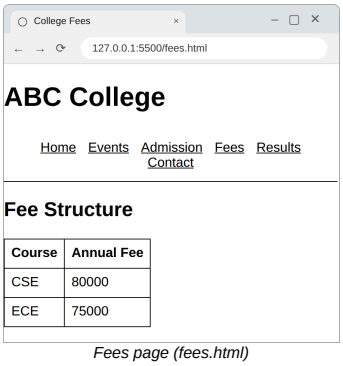
Home page (index.html)



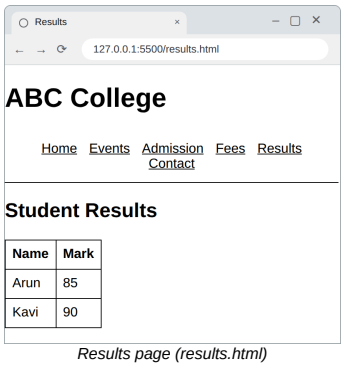
Events page (events.html)



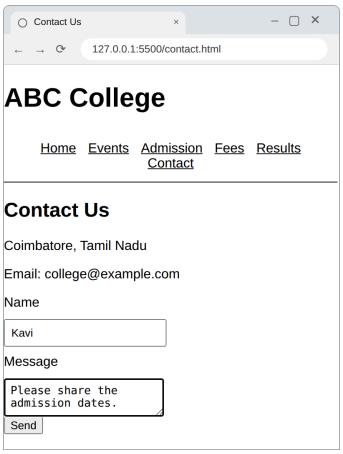
Admission page (admission.html)



Fees page (fees.html)



Results page (results.html)



Contact page (contact.html) with feedback form

Experiment 6 — Shopping Webpage with Navigation and Dropdown

Concept: Navigation links, nested lists/menus, CSS positioning and the `:hover` selector.

index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Simple Mart</title>
    <link rel="stylesheet" href="styles.css">
</head>
<body>
    <header>
        <h1>Simple Mart</h1>
    </header>

    <nav>
        <a href="#">Home</a>

        <div class="dropdown">
            <a href="#">Products</a>
            <div class="menu">
                <a href="#grocery">Groceries</a>
                <a href="#fruit">Fruits</a>
                <a href="#dairy">Dairy</a>
            </div>
        </div>

        <a href="#offers">Offers</a>
        <a href="#contact">Contact</a>
    </nav>

    <h2 class="center">Our Products</h2>

    <section class="products">
        <div id="grocery" class="card">
            <h3>Groceries</h3>
            <p>Rice and pulses</p>
        </div>

        <div id="fruit" class="card">
            <h3>Fruits</h3>
            <p>Fresh fruits</p>
        </div>

        <div id="dairy" class="card">
            <h3>Dairy</h3>
            <p>Milk and cheese</p>
        </div>
    </section>

    <p id="offers" class="center">Special offers available.</p>
    <p id="contact" class="center">Contact: shop@example.com</p>
</body>
</html>
```

styles.css

```
body {
    font-family: Arial;
    margin: 0;
    color: black;
    background: white;
}

header, .center {
    text-align: center;
}

header {
    border-bottom: 1px solid black;
}

nav {
    display: flex;
    justify-content: center;
    gap: 25px;
    padding: 12px;
    border-bottom: 1px solid black;
}

nav a {
    color: black;
    text-decoration: none;
}

.dropdown {
    position: relative;
}

.menu {
    display: none;
    position: absolute;
    background: white;
    border: 1px solid black;
    min-width: 120px;
}

.menu a {
    display: block;
    padding: 8px;
}

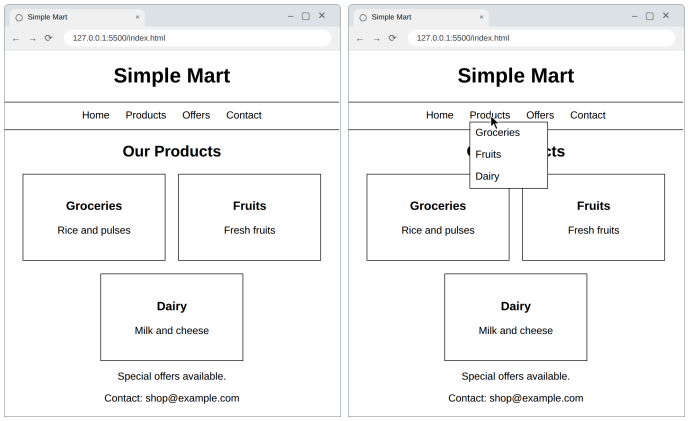
.dropdown:hover .menu {
    display: block;
}

.products {
    display: flex;
    justify-content: center;
    gap: 20px;
    flex-wrap: wrap;
}

.card {
    border: 1px solid black;
    padding: 20px;
    width: 180px;
    text-align: center;
}
```

Understand: `display: none` hides the dropdown. When the pointer hovers over the parent element, `.dropdown:hover .menu` changes it to `display: block`. `position: absolute` positions the dropdown relative to its positioned parent.

OUTPUT



Output 1 — shopping page *Output 2 — hovering over Products opens the dropdown*

Experiments 7(a)–7(e): JavaScript Programs

These five programs can be saved separately as `.js` files and executed with Node.js. Open a terminal in the experiment folder and run the corresponding command, such as `node 7a.js`.

Experiment 7(a) — Electricity Bill Calculation

Concept: `if`, `else if`, `else` and arithmetic operations.

7a.js

```
let units = 250;
let bill;

if (units <= 100) {
    bill = units * 2;
} else if (units <= 200) {
    bill = 100 * 2 + (units - 100) * 3;
} else {
    bill = 100 * 2 + 100 * 3 + (units - 200) * 5;
}

console.log("Units:", units);
console.log("Electricity Bill: Rs.", bill);
```

Output:

```
Units: 250
Electricity Bill: Rs. 950
```

Understand: The bill uses progressive slabs: the first 100 units cost ₹2 per unit, the next 100 cost ₹3, and units above 200 cost ₹5.

OUTPUT

```
TERMINAL  PROBLEMS  OUTPUT  DEBUG CONSOLE  powershell ▾

PS D:\WT\Ex07> node 7a.js
Units: 250
Electricity Bill: Rs. 750
```

Console output — node 7a.js

Experiment 7(b) — Student Marks Analysis

Concept: Arrays, loops, total, average, maximum and minimum.

7b.js

```
let marks = [85, 78, 92, 88, 76];
let total = 0;

for (let i = 0; i < marks.length; i++) {
    total += marks[i];
}

let average = total / marks.length;
let highest = Math.max(...marks);
let lowest = Math.min(...marks);

console.log("Marks:", marks.join(", "));
console.log("Total:", total);
console.log("Average:", average);
console.log("Highest:", highest);
console.log("Lowest:", lowest);
```

Output:

```
Marks: 85, 78, 92, 88, 76
Total: 419
Average: 83.8
Highest: 92
Lowest: 76
```

Understand: `marks.length` gives the number of marks. The loop adds each mark to `total`. The spread operator `...marks` passes the array elements individually to `Math.max()` and `Math.min()`.

OUTPUT

```
TERMINAL  PROBLEMS  OUTPUT  DEBUG CONSOLE  powershell ▾

PS D:\WT\Ex07> node 7b.js
Marks: 85, 78, 92, 88, 76
Total: 419
Average: 83.8
Highest: 92
Lowest: 76
```

Console output — node 7b.js

Experiment 7(c) — ATM PIN Verification

Concept: `while` loop, conditional statements, user input and `break`.

7c.js

```
const readline = require("node:readline");

const rl = readline.createInterface({
    input: process.stdin,
    output: process.stdout
});
```

```
let correctPIN = "2920";
let attempts = 0;

function verifyPIN() {
    rl.question("Enter PIN: ", function(pin) {
        attempts++;

        if (pin === correctPIN) {
            console.log("PIN Verified");
            rl.close();
        } else if (attempts === 3) {
            console.log("Wrong PIN Attempt");
            console.log("ATM Blocked");
            rl.close();
        } else {
            console.log("Wrong PIN Attempt");
            verifyPIN();
        }
    });
}

verifyPIN();
```

Example output for three incorrect attempts:

```
Enter PIN: 1111
Wrong PIN Attempt
Enter PIN: 2222
Wrong PIN Attempt
Enter PIN: 3333
Wrong PIN Attempt
ATM Blocked
```

Understand: The function calls itself again when the PIN is incorrect and fewer than three attempts have been used. This is recursion. The program stops accepting input when the PIN is correct or the third attempt fails.

OUTPUT

TERMINAL PROBLEMS OUTPUT DEBUG CONSOLE powershell ▾

```
PS D:\WT\Ex07> node 7c.js
Enter PIN: 1111
Wrong PIN Attempt
Enter PIN: 2222
Wrong PIN Attempt
Enter PIN: 3333
Wrong PIN Attempt
ATM Blocked

PS D:\WT\Ex07> node 7c.js
Enter PIN: 2920
PIN Verified
```

Console output — node 7c.js (three wrong PINs, then correct PIN)

Experiment 7(d) — Shopping Purchase Amount

Concept: Arrays, loops, addition and conditional statements.

7d.js

```
let prices = [500, 750, 250, 1000];
let total = 0;

for (let i = 0; i < prices.length; i++) {
    total += prices[i];
}

console.log("Prices:", prices.join(", "));
console.log("Total Amount: Rs.", total);

if (total >= 2000) {
    console.log("Eligible for Discount");
} else {
    console.log("Not Eligible for Discount");
}
```

Output:

```
Prices: 500, 750, 250, 1000
Total Amount: Rs. 2500
Eligible for Discount
```

Understand: The loop calculates the purchase total. The condition checks whether the amount meets the discount threshold of ₹2,000.

OUTPUT

TERMINAL PROBLEMS OUTPUT DEBUG CONSOLE powershell ▾

```
PS D:\WT\Ex07> node 7d.js
Prices: 500, 750, 250, 1000
Total Amount: Rs. 2500
Eligible for Discount
```

Console output — node 7d.js

Experiment 7(e) — Employee Annual Salary

Concept: Arrays of objects, object properties and loops.

7e.js

```
let employees = [
    { name: "Arun", salary: 25000 },
    { name: "Bala", salary: 30000 },
    { name: "Charan", salary: 28000 }
];

for (let i = 0; i < employees.length; i++) {
    let annual = employees[i].salary * 12;

    console.log("Name:", employees[i].name);
    console.log("Monthly Salary:", employees[i].salary);
    console.log("Annual Salary:", annual);
    console.log("--------------------");
}
```

Understand: Each employee is an object containing `name` and `salary`. The expression `employees[i].salary * 12` calculates the annual salary.

OUTPUT

TERMINAL PROBLEMS OUTPUT DEBUG CONSOLE powershell ▾

```
PS D:\WT\Ex07> node 7e.js
Name: Arun
Monthly Salary: 25000
Annual Salary: 300000
--------------------
Name: Bala
Monthly Salary: 30000
Annual Salary: 360000
--------------------
Name: Charan
Monthly Salary: 28000
Annual Salary: 336000
--------------------
```

Console output — node 7e.js

Experiments 8(a)–8(f): JavaScript DOM

The DOM (Document Object Model) allows JavaScript to access and modify HTML elements. For these experiments, each `index.html` is a complete webpage, with styling in a separate `styles.css` file.

Experiment 8(a) — Mouse Events on a Table

Concept: `mouseover`, `mouseout`, `click`, `dblclick` and `classList`.

index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Table Mouse Events</title>
    <link rel="stylesheet" href="styles.css">
</head>
<body>
    <h2>Table Mouse Events</h2>
    <p>Hover, click, or double-click a row.</p>

    <table id="myTable">
        <tr><th>Name</th><th>Department</th></tr>
        <tr><td>Arun</td><td>CSE</td></tr>
        <tr><td>Rahul</td><td>ECE</td></tr>
        <tr><td>Kavi</td><td>IT</td></tr>
    </table>

    <script>
        let rows = document.querySelectorAll(
            "#myTable tr:not(:first-child)"
        );

        rows.forEach(function(row) {
            row.onmouseover = function() {
                row.style.backgroundColor = "lightgray";
            };

            row.onmouseout = function() {
                if (!row.classList.contains("selected")) {
                    row.style.backgroundColor = "";
                }
            };

            row.onclick = function() {
                row.classList.toggle("selected");
                row.style.backgroundColor =
                    row.classList.contains("selected") ? "gray" :
"";
            };

            row.ondblclick = function() {
                row.classList.remove("selected");
```

```
            row.style.backgroundColor = "";
        };
    });
    </script>
</body>
</html>
```

styles.css

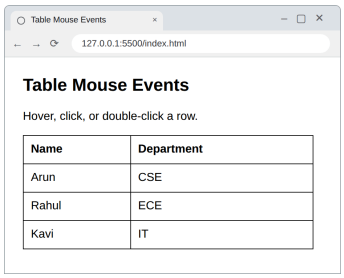
```
body {
    font-family: Arial;
    margin: 25px;
}

table {
    border-collapse: collapse;
    width: 400px;
}

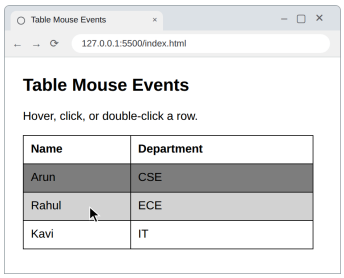
th, td {
    border: 1px solid black;
    padding: 10px;
    text-align: left;
}
```

Understand: `querySelectorAll()` selects the table rows except the heading row. `classList.toggle()` adds a class when it is absent and removes it when present.

OUTPUT



Output 1 — initial table



Output 2 — Arun clicked (selected), Rahul hovered

Experiment 8(b) — Real-Time Digital Clock

Concept: JavaScript `Date`, `setInterval()` and updating HTML text.

index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Digital Clock</title>
    <link rel="stylesheet" href="styles.css">
</head>
<body>
    <h2>Real-Time Digital Clock</h2>
    <p id="clock"></p>

    <script>
        function showTime() {
            let now = new Date();

            let h = String(now.getHours()).padStart(2, "0");
            let m = String(now.getMinutes()).padStart(2, "0");
            let s = String(now.getSeconds()).padStart(2, "0");

            document.getElementById("clock").textContent =
                h + ":" + m + ":" + s;
        }

        showTime();
        setInterval(showTime, 1000);
    </script>
</body>
</html>
```

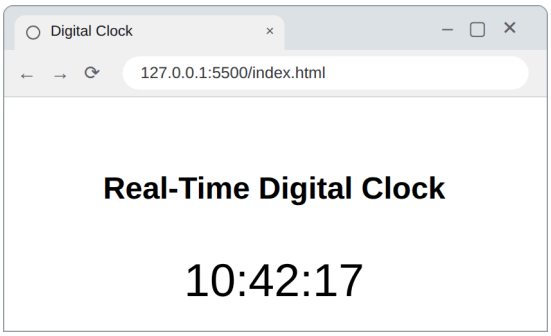
styles.css

```
body {
    font-family: Arial;
    text-align: center;
    padding: 30px;
}

#clock {
    font-size: 36px;
}
```

Understand: `new Date()` gets the current date and time. `setInterval(showTime, 1000)` calls the function every 1,000 milliseconds, which is one second.

OUTPUT



Output — clock updates every second

Experiment 8(c) — Keyboard Interaction

Concept: The `keydown` event, `event.key` and changing CSS using JavaScript.

index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Keyboard Interaction</title>
    <link rel="stylesheet" href="styles.css">
</head>
<body>
    <h2>Keyboard Interaction</h2>
    <p>Press A to change the border.</p>
    <p>Press R to reset the border.</p>

    <table id="myTable">
        <tr><th>Name</th><th>Department</th></tr>
        <tr><td>Arun</td><td>CSE</td></tr>
        <tr><td>Kavi</td><td>IT</td></tr>
    </table>

    <script>
        document.addEventListener("keydown", function(event) {
            let table = document.getElementById("myTable");

            if (event.key.toLowerCase() === "a") {
                table.style.border = "4px solid black";
            }

            if (event.key.toLowerCase() === "r") {
                table.style.border = "1px solid black";
            }
        });
    </script>
</body>
</html>
```

styles.css

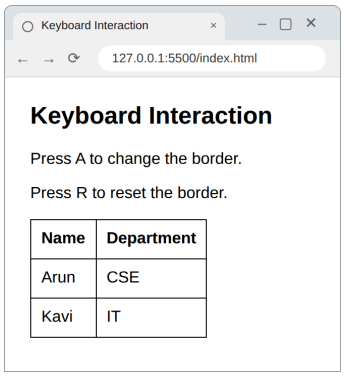
```
body {
    font-family: Arial;
    margin: 25px;
}

table {
    border-collapse: collapse;
    border: 1px solid black;
}

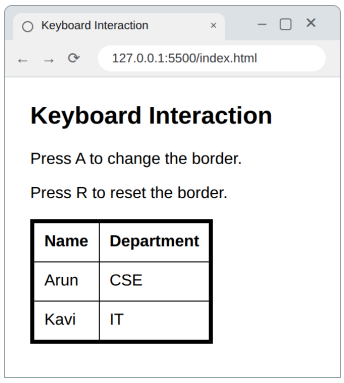
th, td {
    border: 1px solid black;
    padding: 10px;
}
```

Understand: `addEventListener("keydown", ...)` listens for keyboard input. `event.key` identifies the key pressed, and `toLowerCase()` makes the comparison work for either uppercase or lowercase letters.

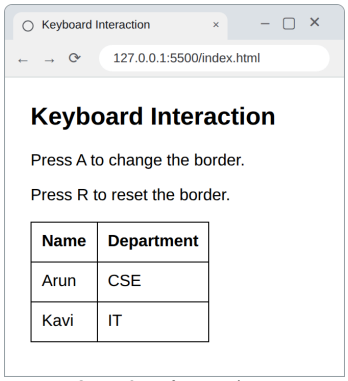
OUTPUT



Output 1 — initial table



Output 2 — after pressing A



Output 3 — after pressing R

Experiment 8(d) — Simple Form with Display

Concept: Reading input values, handling form submission and preventing page reload.

index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Simple Form</title>
    <link rel="stylesheet" href="styles.css">
</head>
<body>
    <h2>Simple Form</h2>

    <form id="myForm">
        <input type="text" id="name"
               placeholder="Enter your name" required>
        <button type="submit">Submit</button>
    </form>

    <p id="output"></p>

    <script>
        document.getElementById("myForm").onsubmit =
            function(event) {
                event.preventDefault();

                let value = document.getElementById("name").value;

                document.getElementById("output").textContent =
                    "You entered: " + value;
            };
    </script>
</body>
</html>
```

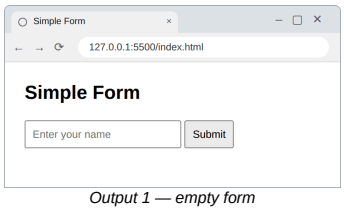
styles.css

```
body {
    font-family: Arial;
    margin: 25px;
}

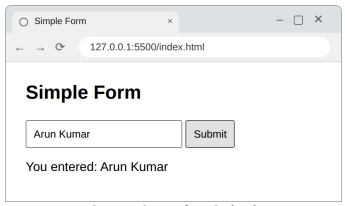
input, button {
    padding: 8px;
}
```

Understand: `.value` reads what the user typed. `textContent` displays the result. `preventDefault()` stops the normal form submission, so the webpage does not reload.

OUTPUT



Output 1 — empty form



Output 2 — after Submit

Experiment 8(e) — Mini Login System

Concept: Reading username/password and checking credentials with `if-else`.

index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Mini Login System</title>
    <link rel="stylesheet" href="styles.css">
</head>
<body>
    <h2>Mini Login System</h2>

    <input type="text" id="username" placeholder="Username">
    <input type="password" id="password" placeholder="Password">

    <button onclick="login()">Login</button>

    <p id="message"></p>

    <script>
        function login() {
            let username =
document.getElementById("username").value;
            let password =
document.getElementById("password").value;

            if (username === "admin" && password === "1234") {
                document.getElementById("message").textContent =
                    "Welcome";
            } else {
                document.getElementById("message").textContent =
                    "Invalid credentials";
            }
        }
    </script>
</body>
</html>
```

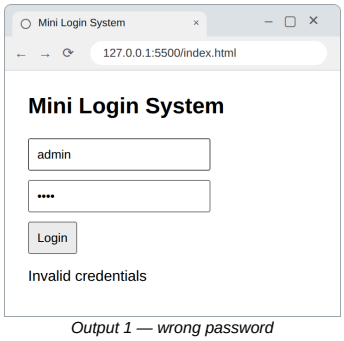
styles.css

```
body {
    font-family: Arial;
    margin: 25px;
}

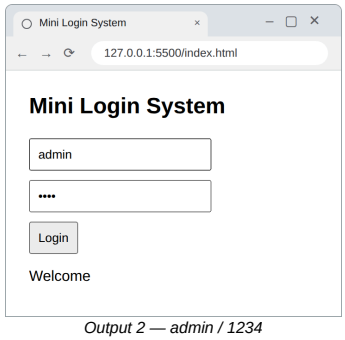
input, button {
    display: block;
    padding: 8px;
    margin: 10px 0;
}
```

Understand: The `&&` operator means both conditions must be true. The username must be `admin` and the password must be `1234`. This is only a demonstration, not secure real-world authentication.

OUTPUT



Output 1 — wrong password



Output 2 — admin / 1234

Experiment 8(f) — Dark/Light Mode

Concept: CSS classes, click events and `classList.toggle()`.

index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Dark Light Mode</title>
    <link rel="stylesheet" href="styles.css">
</head>
<body>
    <h2>Dark / Light Mode</h2>
    <p>Click the button to change the theme.</p>

    <button id="themeBtn">Toggle Theme</button>

    <script>
        document.getElementById("themeBtn").onclick = function() {
            document.body.classList.toggle("dark");
        };
    </script>
</body>
</html>
```

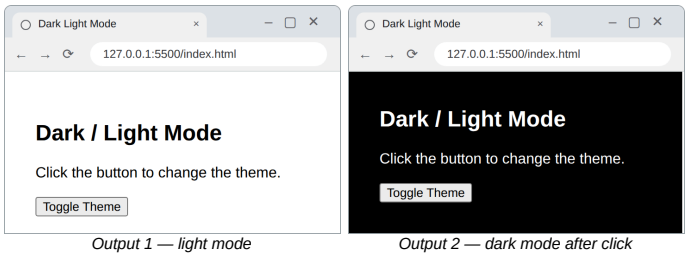
styles.css

```
body {
    font-family: Arial;
    padding: 25px;
    background: white;
    color: black;
}

.dark {
    background: black;
    color: white;
}
```

Understand: Clicking the button adds or removes the `dark` class on the body. CSS then changes the background and text colours automatically.

OUTPUT



Output 1 — light mode *Output 2 — dark mode after click*

Experiment 9 — AngularJS Form Validation

Concept: AngularJS modules, controllers, data binding, form validation and directives.

This experiment uses AngularJS 1.x, matching the subject in your record. Save the following two files in the same folder.

index.html

```
<!DOCTYPE html>
<html ng-app="app">
<head>
    <title>AngularJS Form Validation</title>

    <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular
.min.js"></script>

    <link rel="stylesheet" href="styles.css">
</head>

<body ng-controller="FormCtrl">
    <h2>Registration Form</h2>

    <form name="f" ng-submit="submit()" novalidate>

        <label>Name</label>
        <input name="name" ng-model="user.name" required>
        <p ng-show="f.name.$touched && f.name.$invalid">
            Name is required.
        </p>

        <label>Email</label>
        <input type="email" name="email"
               ng-model="user.email" required>
        <p ng-show="f.email.$touched && f.email.$invalid">
            Enter a valid email.
        </p>

        <label>Password</label>
        <input type="password" name="password"
               ng-model="user.password"
               ng-minlength="4" required>
        <p ng-show="f.password.$touched && f.password.$invalid">
            Password must contain at least 4 characters.
        </p>

        <button type="submit" ng-disabled="f.$invalid">
            Register
        </button>
    </form>

    <p>{{message}}</p>

    <script>
        angular.module("app", [])
        .controller("FormCtrl", function($scope) {
            $scope.submit = function() {
                $scope.message =
                    "Registration successful for " +
$scope.user.name;
            };
        });
    </script>
</body>
</html>
```

styles.css

```
body {
    font-family: Arial;
    margin: 25px;
    color: black;
}

label {
    display: block;
    margin: 12px 0;
}

input {
    display: block;
    padding: 7px;
    margin-top: 4px;
}

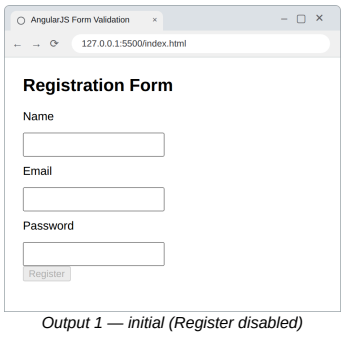
p {
    margin: 8px 0;
}
```

Understand the important directives:

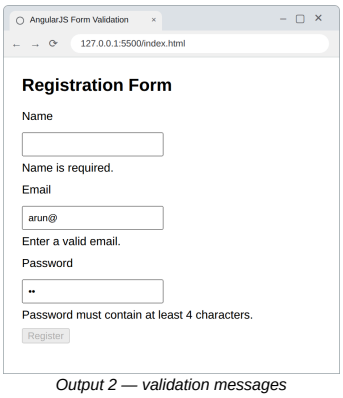
- `ng-app="app"` starts the AngularJS application.
- `ng-controller="FormCtrl"` connects the page to the controller.
- `ng-model="user.name"` binds an input field to a JavaScript model.
- `ng-submit="submit()"` calls a function when the form is submitted.
- `ng-disabled="f.$invalid"` disables the button while the form is invalid.
- `{{message}}` displays a value from the controller.

The AngularJS library must load successfully, so an internet connection is required for this CDN version.

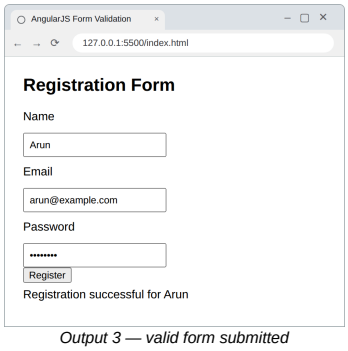
OUTPUT



Output 1 — initial (Register disabled)



Output 2 — validation messages



Output 3 — valid form submitted

Experiment 10 — React Components, Props, State and Events

Concept: Creating a component, passing props, managing state and handling button clicks.

This is a simple React version that can be opened directly in a browser with an internet connection.

index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>React Components</title>
    <link rel="stylesheet" href="styles.css">

    <script
src="https://unpkg.com/react@18/umd/react.development.js">
</script>
    <script src="https://unpkg.com/react-dom@18/umd/react-
dom.development.js"></script>
    <script
src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
</head>
<body>
    <div id="root"></div>

    <script type="text/babel">
        function Greeting(props) {
            return <h2>Hello, {props.name}</h2>;
        }

        function App() {
            const [count, setCount] = React.useState(0);

            return (
                <div>
                    <Greeting name="Student" />

                    <p>Button clicked: {count} times</p>

                    <button onClick={() => setCount(count + 1)}>
                        Click
                    </button>
                </div>
            );
        }

        ReactDOM.createRoot(
            document.getElementById("root")
        ).render(<App />);
    </script>
</body>
</html>
```

styles.css

```
body {
    font-family: Arial;
    margin: 25px;
    color: black;
    background: white;
}

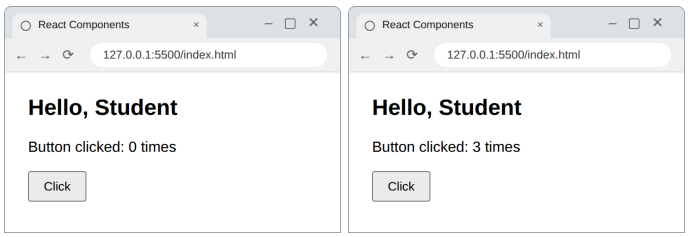
button {
    padding: 7px 15px;
}
```

Understand:

- Component: `Greeting()` is a reusable part of the interface.
- Props: `props.name` receives the value passed through `name="Student"`.
- State: `useState(0)` stores the counter's initial value.
- Event: `onClick` handles a button click.
- JSX: HTML-like syntax written inside JavaScript.

When `setCount()` updates the state, React renders the updated counter.

OUTPUT



Output 1 — initial state *Output 2 — after three clicks*

Experiment 11 — React Component State and Input Handling

Concept: Controlled inputs, `useState()` and the `onChange` event.

index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>React Input Handling</title>
    <link rel="stylesheet" href="styles.css">

    <script
src="https://unpkg.com/react@18/umd/react.development.js">
</script>
    <script src="https://unpkg.com/react-dom@18/umd/react-
dom.development.js"></script>
    <script
src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
</head>
<body>
    <div id="root"></div>

    <script type="text/babel">
        function App() {
            const [name, setName] = React.useState("");

            return (
                <div>
                    <h2>Enter Your Name</h2>

                    <input
                        value={name}
                        onChange={e => setName(e.target.value)}
                        placeholder="Enter your name"
                    />

                    <p>Hello, {name || "Student"}</p>
                </div>
            );
        }

        ReactDOM.createRoot(
            document.getElementById("root")
        ).render(<App />);
    </script>
</body>
</html>
```

styles.css

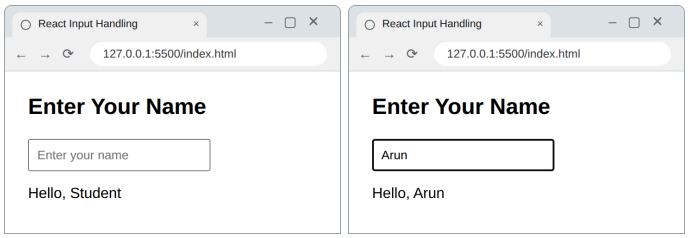
```
body {
    font-family: Arial;
    margin: 25px;
    color: black;
}

input {
    padding: 8px;
}
```

Understand the input flow: When the user types, `onChange` captures the new value. `setName()` updates the state, and React displays the updated name in the paragraph.

The `value={name}` attribute makes this a controlled input because React state determines the input's displayed value.

OUTPUT



Output 1 — empty input *Output 2 — after typing a name*

Experiment 12 — Node.js and MongoDB

Concept: Creating a server, connecting to a database, defining a model, and performing CRUD operations.

CRUD means:

- Create: Insert a student.
- Read: Retrieve student records.
- Update: Modify a student's details.
- Delete: Remove a student.

This experiment requires Node.js and a running MongoDB instance, or a MongoDB Atlas connection. Unlike the earlier HTML experiments, this is a backend program.

Create the following folder structure:

```
Ex12/
    package.json
    server.js
```

package.json

```
{
    "name": "student-crud",
    "version": "1.0.0",
    "main": "server.js",
    "scripts": {
        "start": "node server.js"
    },
    "dependencies": {
        "express": "^4.21.2",
        "mongoose": "^8.9.5"
    }
}
```

server.js

```
const express = require("express");
const mongoose = require("mongoose");

const app = express();
app.use(express.json());

// Connect to MongoDB
mongoose.connect("mongodb://127.0.0.1:27017/studentdb")
    .then(() => console.log("MongoDB connected"))
    .catch(err => console.log(err.message));

// Create student model
const Student = mongoose.model("Student", {
    name: { type: String, required: true },
    mark: { type: Number, required: true }
});

// Home page
app.get("/", (req, res) => {
    res.send("Student CRUD API is running");
});

// CREATE
app.post("/students", async (req, res) => {
    try {
        const student = await Student.create(req.body);
        res.status(201).json(student);
    } catch (err) {
        res.status(400).json({ error: err.message });
    }
});

// READ ALL
app.get("/students", async (req, res) => {
    const students = await Student.find();
    res.json(students);
});

// READ ONE
app.get("/students/:id", async (req, res) => {
    try {
        const student = await Student.findById(req.params.id);

        if (!student) {
            return res.status(404).json({ error: "Not found" });
        }

        res.json(student);
    } catch {
        res.status(400).json({ error: "Invalid ID" });
    }
});

// UPDATE
app.put("/students/:id", async (req, res) => {
    try {
        const student = await Student.findByIdAndUpdate(
            req.params.id,
            req.body,
            { new: true, runValidators: true }
        );

        if (!student) {
            return res.status(404).json({ error: "Not found" });
        }

        res.json(student);
    } catch (err) {
        res.status(400).json({ error: err.message });
    }
});

// DELETE
app.delete("/students/:id", async (req, res) => {
    try {
        const student = await
Student.findByIdAndDelete(req.params.id);

        if (!student) {
            return res.status(404).json({ error: "Not found" });
        }

        res.json({ message: "Student deleted" });
    } catch {
        res.status(400).json({ error: "Invalid ID" });
    }
});

// Start server
app.listen(3000, () => {
    console.log("Server running at http://localhost:3000");
});
```

3. How to run it

Your folder should look like this:

```
Ex12/
├── package.json
└── server.js
```

Open the terminal inside `Ex12` and execute:

```
npm install
npm start
```

Ensure MongoDB is running locally before starting the program.

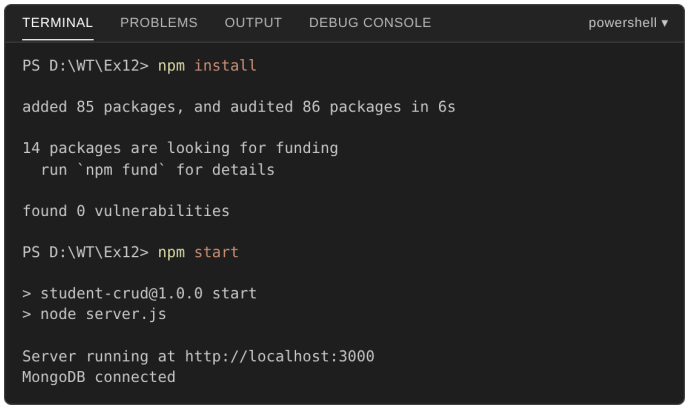
Then open `http://localhost:3000` in your browser.

You should see:

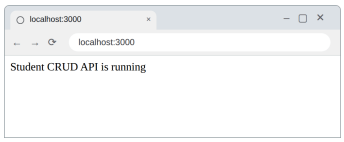
```
Student CRUD API is running
```

Remember for the exam: `POST` creates, `GET` reads, `PUT` updates and `DELETE` removes records. `express.json()` reads JSON request bodies, and Mongoose communicates with MongoDB.

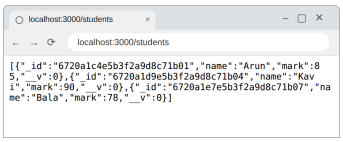
OUTPUT



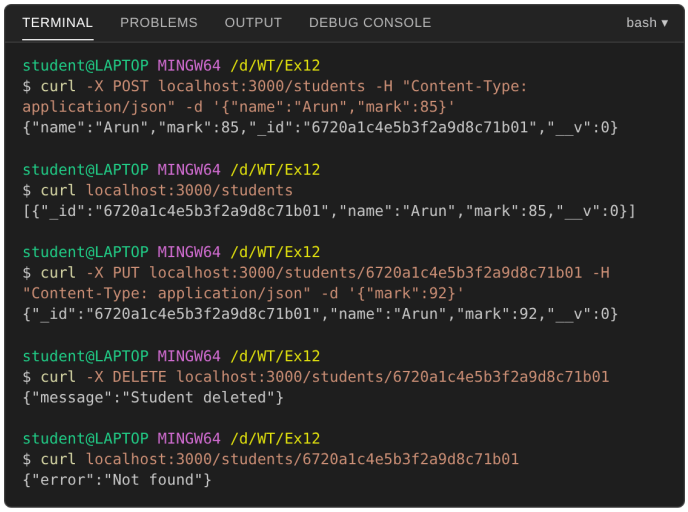
Terminal — npm install and npm start



Browser — http://localhost:3000



Browser — http://localhost:3000/students



Terminal — CRUD requests: POST, GET, PUT, DELETE, GET (curl)